

0811 데이터 파이프라인 정리

SHREC + FPHA 파이프라인 정리 (EDA · 전처리 · 파이프라인 구성)

1. 개요

SHREC'17을 벤치마크로 고정하고, 학습 데이터로 FPHA를 추가하여 EDA부터 학습 직전 단계(Dataset/DataLoader)까지 파이프라인을 구축·검증함. 관절 개수가 다른 두 데이터셋 (SHREC 22개, FPHA 21개)을 통합 좌표계로 매핑하고 공통 전처리를 적용한 뒤, occlusion 샘플을 생성해 실제로 배치가 정상적으로 나오는 것까지 확인 완료.

2. EDA

2-1. 인덱스 테이블 구축

두 데이터셋의 전체 시퀀스를 스캔해 메타데이터(참가자·동작·프레임 수 등)와 파일 경로를 표로 정리함.

```
# SHREC 인덱스 테이블 (2800개 시퀀스)
shrec_root = "/content/shrec2017/HandGestureDataset_SHREC2017"
records = []
for skel_path in glob.glob(f"{shrec_root}/gesture_*/finger_*/subject_*/essai_*/skeletons_world.txt"):
    parts = skel_path.split("/")
    gesture = int(parts[-5].replace("gesture_", ""))
    finger = int(parts[-4].replace("finger_", ""))
    subject = int(parts[-3].replace("subject_", ""))
    essai = int(parts[-2].replace("essai_", ""))
    n_frames = sum(1 for _ in open(skel_path))
    records.append({"gesture": gesture, "finger": finger,
                    "subject": subject,
                    "essai": essai, "n_frames": n_frames,
                    "path": skel_path})
df = pd.DataFrame(records)

# FPHA 인덱스 테이블 (1175개 시퀀스, 공식 제외 대상 3개 필터링)
```

```

fpha_root = "/content/fpha/Hand_pose_annotation_v1"
exclude = {("Subject_2", "close_milk", "4"), ("Subject_2", "put_tea_bag", "2"), ("Subject_4", "flip_sponge", "2")}
records = []
for skel_path in glob.glob(f"{fpha_root}/Subject_*/**/*/skeleton.txt"):
    parts = skel_path.split("/")
    subject_dir, action, rep = parts[-4], parts[-3], parts[-2]
    if (subject_dir, action, rep) in exclude:
        continue
    subject = int(subject_dir.replace("Subject_", ""))
    n_frames = sum(1 for _ in open(skel_path))
    records.append({"subject": subject, "action": action, "rep": int(rep), "n_frames": n_frames, "path": skel_path})
df_fpha = pd.DataFrame(records)

```

결과: SHREC 2,800개, FPHA 1,175개(공식 문서 수치와 일치) 확인.

2-2. 시퀀스 길이 분포

	SHREC	FPHA
평균	58.9프레임	89.8프레임
범위	7~171	7~1151 (편차 큼)
중앙값	57	77

FPHA가 전반적으로 더 길고 이상치(최대 1151프레임)도 있으나, occlusion 샘플링 조건 (N=15, 최대 L=30, 즉 T≥45)에는 두 데이터셋 모두 문제 없음을 확인함.

2-3. 가림 구간 움직임량 비교

FPHA의 가림 구간 움직임을 확인하고자 직접 측정함. 30프레임 구간 내 전체 관절의 프레임 간 이동거리 합을 슬라이딩 방식으로 계산.

```

def window_motion_stats(seq_pre, window=30):
    diffs = np.linalg.norm(np.diff(seq_pre, axis=0), axis=2)
    per_frame_motion = diffs.sum(axis=1)

```

```

if len(per_frame_motion) < window:
    return None
return np.convolve(per_frame_motion, np.ones(window), mode='valid')

```

결과: FPHA 평균 26.9 vs SHREC 평균 75.7 — FPHA가 SHREC 대비 약 1/3 수준의 움직임만 가짐. SHREC 제스처는 크고 확실한 스와이프·회전 동작인 반면, FPHA의 생활 동작 (소금 뿌리기, 계산기 누르기 등)은 손목이 고정된 채 손가락만 미세하게 움직이는 경우가 많아 발생하는 차이로 추정됨. 이 결과에 따라 FPHA는 단독 사용이 아닌 SHREC과 혼합 사용하는 방향으로 파이프라인을 설계함.

3. 전처리

3-1. 문제: 관절 개수 불일치 (22 vs 21)

SHREC 22관절과 FPHA 21관절은 해부학적 이름 기준으로 정확히 대응됨(SHREC의 손바닥 중심점만 FPHA에 대응 관절이 없어 제외). FPHA 순서를 통합 기준으로 채택.

```

shrec_to_unified = {
    0: 0,
    2: 1, 3: 6, 4: 7, 5: 8,          # 엄지
    6: 2, 7: 9, 8: 10, 9: 11,       # 검지
    10: 3, 11: 12, 12: 13, 13: 14,  # 중지
    14: 4, 15: 15, 16: 16, 17: 17,  # 약지
    18: 5, 19: 18, 20: 19, 21: 20,  # 소지
}

def shrec_to_unified_joints(seq_22):
    idx_shrec = sorted(shrec_to_unified.keys())
    idx_unified = [shrec_to_unified[i] for i in idx_shrec]
    unified = np.zeros((seq_22.shape[0], 21, 3))
    unified[:, idx_unified, :] = seq_22[:, idx_shrec, :]
    return unified

```

3-2. 정규화 — 손목 기준 상대좌표 + 스케일 정규화

손바닥 관절이 빠졌으므로, 기존(손목-손바닥 거리) 대신 **손목-중지MCP 거리**를 스케일 기준으로 사용.

```
def load_and_preprocess_unified(seq_21):
    seq = seq_21 - seq_21[:, 0:1, :] #
    손목(0) 기준 상대좌표
    scale = np.linalg.norm(seq[:, 3, :], axis=1).mean() #
    손목-중지MCP(3) 거리로 정규화
    return seq / (scale + 1e-8)
```

이 정규화 덕분에 SHREC·FPHA의 좌표 단위가 다르더라도(정확히 검증되진 않았으나 둘 다 mm로 추정) 손 크기 기준의 상대값으로 변환되어 문제되지 않음.

3-3. 검증

수치 검증(전처리 후 손목 좌표가 (0,0,0), 손목-중지MCP 거리가 1.0에 근접)과 3D 시각화(손가락 5개가 손목에서 갈라져 나가는 형태 확인) 두 가지 방식으로 전처리 로직과 관절 매핑이 올바른지 확인 완료.

4. 파이프라인 구성

4-1. Train / Val / Test 분할

SHREC 벤치마크를 고정함.

essai(반복 시도) 단위로 나눌 경우 동일 인물의 다른 시도가 train/test에 섞여 데이터 누수가 발생하기 때문에 subject 단위로 분할.

```
SHREC_TEST_SUBJECTS = {25, 26, 27, 28}
df_shrec_test = df[df["subject"].isin(SHREC_TEST_SUBJECTS)]
df_shrec_train = df[~df["subject"].isin(SHREC_TEST_SUBJECTS)]

FPHA_VAL_SUBJECTS = {5, 6}
df_fpha_val = df_fpha[df_fpha["subject"].isin(FPHA_VAL_SUBJECTS)]
df_fpha_train = df_fpha[~df_fpha["subject"].isin(FPHA_VAL_SUBJECTS)]
```

결과: SHREC train 2,240 / test(벤치마크) 560, FPHA train 761 / val 414.

4-2. Occlusion 샘플 생성

기존 SHREC 단일 데이터셋용으로 만들었던 `make_occlusion_sample` 로직을 그대로 재사용 하되, 통합 좌표계로 감싼 로더 함수(`load_shrec_pre`, `load_fpha_pre`)만 바꿔 끼워 두 데이터 셋에 동일하게 적용함. N=15(입력), L=5/10/20/30(가림 길이).

```
def make_occlusion_sample(seq, N, L, seed=None):
    rng = np.random.default_rng(seed)
    T = seq.shape[0]
    if T < N + L:
        return None
    start = rng.integers(N, T - L + 1)
    return seq[start-N:start], seq[start:start+L]

def build_dataset(df_in, loader_fn, source_name, N=15, L_list=(5, 10, 20, 30)):
    dataset = []
    for _, row in df_in.iterrows():
        seq = loader_fn(row["path"])
        for L in L_list:
            result = make_occlusion_sample(seq, N=N, L=L)
            if result is None:
                continue
            input_seq, target_seq = result
            dataset.append({"source": source_name, "L": L,
                "input": input_seq, "target": target_seq})
    return dataset

train_samples = build_dataset(df_fpha_train, load_fpha_pre,
    "fpha") + build_dataset(df_shrec_train, load_shrec_pre, "shrec")
val_samples = build_dataset(df_fpha_val, load_fpha_pre, "fpha")
test_samples = build_dataset(df_shrec_test, load_shrec_pre,
    "shrec") # 벤치마크 전용, 학습에 사용 금지
```

최종 산출물: train 10,633개 / val 1,402개 / test(벤치마크) 1,479개.

4-3. Dataset / DataLoader

L(가림 길이)이 섞여있으면 배치 텐서 크기가 안 맞아 L별로 DataLoader를 따로 구성함.

```

class OcclusionDataset(Dataset):
    def __init__(self, samples): self.samples = samples
    def __len__(self): return len(self.samples)
    def __getitem__(self, idx):
        s = self.samples[idx]
        return (torch.tensor(s["input"], dtype=torch.float32),
                torch.tensor(s["target"], dtype=torch.float32))

loaders_by_L = {
    L: DataLoader(OcclusionDataset([s for s in train_sample
s if s["L"] == L]), batch_size=32, shuffle=True)
    for L in (5, 10, 20, 30)
}

```

배치 shape 확인 결과 `input: (32, 15, 21, 3)`, `target: (32, L, 21, 3)` 으로 정상 동작

5. SILK 논문 활용 방안

SILK(2025, motion in-betweening 최신 연구)의 설계 원칙을 위 파이프라인·다음 단계 모델 설계에 아래와 같이 반영할 수 있음.

SILK 발견	파이프라인 반영 상태 / 적용 방안
인코더 단일 구조로 충분	미반영 — LSTM/GRU seq2seq보다 Transformer 인코더 단일 구조로 L프레임 일괄 예측하는 베이스라인 우선 검토
상대적 위치 인코딩만으로 충분	미반영 — 위와 함께 아키텍처 설계 시 적용
빈 프레임은 0으로 채우기	미반영 — 현재는 forecasting 형태(N프레임만 입력)라 해당 없음. in-betweening으로 과제를 확장할 경우 적용 대상
쪼쪼한 슬라이딩 샘플링으로 샘플 수 확보	미반영 — 현재 <code>build_dataset</code> 은 시퀀스당 L별 1개만 샘플링. 아래 코드로 대체 시 샘플 수 대폭 증가 가능
루트(손목) 기준 좌표 표현	반영됨 — <code>load_and_preprocess_unified</code> 에서 손목 기준 상대좌표 사용 중
속도(velocity)를 입력 특징으로만 포함	미반영 — <code>OcclusionDataset</code> 에 속도(프레임 간 차분) 특징 추가 검토, 예측 정답에는 미포함
L1 손실 사용	미반영 — 모델 학습 코드 작성 시 <code>nn.L1Loss()</code> 채택

충족한 샘플링 적용 예시 (발견 ④, 데이터 양 부족 문제의 직접적 해결책):

```
def make_occlusion_samples_dense(seq, N, L, stride=5):
    T = seq.shape[0]
    return [(seq[s-N:s], seq[s:s+L]) for s in range(N, T -
L + 1, stride)]
```

단, SILK에서도 이 효과는 단순 모델(SILK)에서 뚜렷했고 복잡한 모델(TST)에서는 과적합이 발생했다는 점을 감안해, 베이스라인부터 적용 후 효과를 검증할 것을 권장함.

6. 세 번째 데이터셋 추가 시

FPHA를 추가했던 과정을 일반화하면: ① 공식 문서에서 관절 순서 확인 후 해부학적 대응표 작성 ② 기존 `load_and_preprocess_unified` 를 그대로 재사용하고 데이터셋 전용 로더만 추가 작성 ③ 프레임 길이 분포와 가림 구간 움직임량을 EDA로 실측해 SHREC 대비 지나치게 정적이지 않은지 확인 ④ 새 데이터셋은 학습 전용으로만 사용하고 벤치마크(SHREC subject 25~28)는 그대로 유지. DHG-14/28은 SHREC과 형식이 유사해 통합이 수월할 것으로 예상됨.

7. 참고 링크

- [FPHA 공식 GitHub](#)
- [SILK 논문 \(arXiv\)](#).